

Ex.No:5	
Date:	

AIM:

ALGORITHM:

CODE:

```
import hashlib
```

```
def power(a, b, m):
```

```
    r = 1
```

```
    for i in range(b):
```

```
        r = (r * a) % m
```

```
    return r
```

```
def inverse(a, m):  
    for i in range(1, m):  
        if (a * i) % m == 1:  
            return i  
    return 0
```

```
def hash_message(msg, q):  
    h = hashlib.sha256(msg.encode()).digest()  
  
    value = 0  
  
    for b in h:  
        value = (value * 256 + b) % q  
  
    return value
```

```
def main():  
  
    # DSA Parameters  
  
    p = 23  
    q = 11  
    g = 4  
  
    privateKey = 3
```

```
publicKey = power(g, privateKey, p)
```

```
k = 7
```

```
message = input("Enter the message: ")
```

```
h = hash_message(message, q)
```

```
# Signature Generation
```

```
r = power(g, k, p) % q
```

```
s = (inverse(k, q) * (h + privateKey * r)) % q
```

```
print("\n--- DSA Parameters ---")
```

```
print("p =", p)
```

```
print("q =", q)
```

```
print("g =", g)
```

```
print("\n--- Keys ---")
```

```
print("Private Key =", privateKey)
```

```
print("Public Key =", publicKey)
```

```
print("\n--- Signature ---")
```

```
print("Hash =", h)
```

```
print("r =", r)
```

```
print("s =", s)
```

```
print("Signature = (" + str(r) + "," + str(s) + ")")
```

```
# Signature Verification
```

```
w = inverse(s, q)
```

```
u1 = (h * w) % q
```

```
u2 = (r * w) % q
```

```
v = (power(g, u1, p) * power(publicKey, u2, p) % p) % q
```

```
print("\n--- Verification ---")
```

```
print("u1 =", u1)
```

```
print("u2 =", u2)
```

```
print("v =", v)
```

```
if v == r:
```

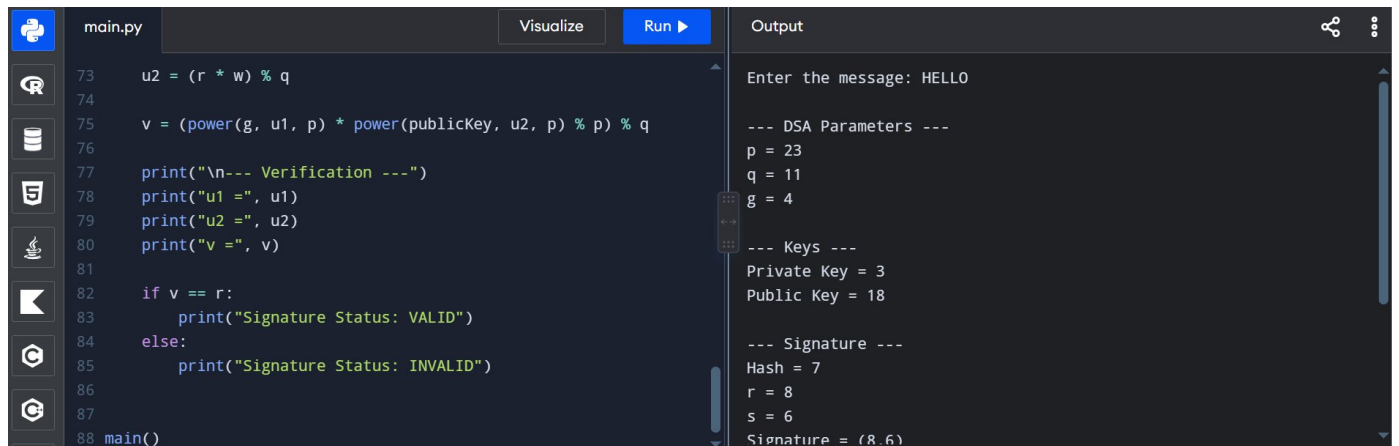
```
    print("Signature Status: VALID")
```

```
else:
```

```
    print("Signature Status: INVALID")
```

```
main()
```

Output:



The screenshot displays a Jupyter Notebook environment. The left sidebar contains standard icons for file management, search, and other notebook functions. The main area is split into two panes. The left pane, titled 'main.py', shows a Python script for verifying a digital signature using the Digital Signature Algorithm (DSA). The script includes calculations for u_2 and v , followed by a verification step that prints 'Signature Status: VALID' or 'INVALID'. The right pane, titled 'Output', shows the execution results, including the input message 'HELLO', DSA parameters ($p=23$, $q=11$, $g=4$), keys (Private Key = 3, Public Key = 18), and the final signature $(8, 6)$.

```
main.py Visualize Run ▶
```

```
73 u2 = (r * w) % q
74
75 v = (power(g, u1, p) * power(publicKey, u2, p) % p) % q
76
77 print("\n--- Verification ---")
78 print("u1 =", u1)
79 print("u2 =", u2)
80 print("v =", v)
81
82 if v == r:
83     print("Signature Status: VALID")
84 else:
85     print("Signature Status: INVALID")
86
87
88 main()
```

```
Output
```

```
Enter the message: HELLO

--- DSA Parameters ---
p = 23
q = 11
g = 4

--- Keys ---
Private Key = 3
Public Key = 18

--- Signature ---
Hash = 7
r = 8
s = 6
Signature = (8,6)
```

Result: